



Terraform Engineering Bootcamp

Week 1 Practical Demonstration Guide

Terraform Foundations — First Infrastructure Deployment

Instructor: Mthulisi Zulu

Role: Cloud Governance Architect

Founder | MZ-UCA Cloud Academy

Practical Objective

By the end of this demonstration, students will understand how Terraform manages infrastructure from code.

We will demonstrate:

- ✓ Creating a Terraform project
- ✓ Installing a provider
- ✓ Writing Terraform configuration
- ✓ Initializing Terraform
- ✓ Validating configuration
- ✓ Creating an execution plan
- ✓ Deploying infrastructure
- ✓ Understanding Terraform state
- ✓ Updating infrastructure
- ✓ Destroying infrastructure

Demonstration Scenario

Business Requirement

"Imagine MZ-UCA Cloud Academy needs an automated documentation system."

Instead of manually creating files, we want Terraform to create and manage them."

Today Terraform will create:

```
welcome.txt
```

```
README.md
```

```
company-policy.txt
```

Part 1 — Create Terraform Project

Instructor Action

Open terminal.

Create project:

```
mkdir terraform-foundations
```

Enter directory:

```
cd terraform-foundations
```

Open VS Code:

```
code .
```

Explain To Students

"Every Terraform project starts with a working directory.

Terraform reads all configuration files ending with:

`.tf`

inside this directory."

Part 2 — Create Terraform Configuration

Create:

```
main.tf
```

Paste:

```
terraform {  
  required_providers {  
    local = {  
      source  = "hashicorp/local"  
      version = "~> 2.5"  
    }  
  }  
}  
  
provider "local" {  
  
}  
  
resource "local_file" "welcome" {  
  filename = "welcome.txt"  
  
  content = <<EOF  
Welcome to Terraform Engineering  
  
MZ-UCA Cloud Academy  
  
Instructor:  
Mthulisi Zulu  
  
Cloud Governance Architect  
  
EOF  
}
```

Explain The Configuration

Terraform Block

```
terraform {  
  
}
```

Explanation:

"This defines Terraform requirements."

Provider

```
local = {  
  source = "hashicorp/local"  
}
```

Explanation:

"A provider is a plugin that allows Terraform to communicate with a platform."

Examples:

```
AWS Provider  
Azure Provider  
Docker Provider  
Kubernetes Provider  
Local Provider
```

Resource

```
resource "local_file" "welcome"
```

Explain:

Every resource has:

```
resource TYPE NAME
```

Example:

Type:

```
local_file
```

Meaning:

Create a file.

Name:

```
welcome
```

Terraform's internal reference.

Part 3 — Initialize Terraform

Run:

```
terraform init
```

Expected Result

```
Terraform has been successfully initialized!
```

Explain

"Terraform has downloaded the provider required for this project."

Show:

```
.terraform/
```

Explain:

"This folder contains provider plugins."

Show:

```
.terraform.lock.hcl
```

Explain:

"This locks the provider version."

Part 4 — Format Configuration

Run:

```
terraform fmt
```

Explain:

"Terraform code should follow standards.

Formatting improves readability and teamwork."

Part 5 — Validate Configuration

Run:

```
terraform validate
```

Expected:

```
Success! The configuration is valid.
```

Explain:

"Terraform is checking whether our code is correct."

Part 6 — Create Execution Plan

Run:

```
terraform plan
```

Expected:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Pause And Ask Students

Question:

"Has Terraform created the file yet?"

Answer:

"No."

Explain:

"Plan only shows us what Terraform wants to do.

It does not make changes."

Part 7 — Deploy Infrastructure

Run:

```
terraform apply
```

Terraform asks:

```
Do you want to perform these actions?
```

Type:

```
yes
```

Expected:

```
Apply complete!  
Resources: 1 added
```

Verify Deployment

Run:

```
dir
```

Students should see:

```
welcome.txt
```

Open:

```
welcome.txt
```

Expected:

```
Welcome to Terraform Engineering
```

```
MZ-UCA Cloud Academy
```

Instructor:
Mthulisi Zulu

Cloud Governance Architect

Teaching Moment

Ask:

"Did we manually create this file?"

Students:

"No."

Explain:

"This is Infrastructure as Code.

The configuration created the resource."

Part 8 — Explain Terraform State

Run:

```
dir
```

Show:

```
terraform.tfstate
```

Explain:

"Terraform needs memory.

The state file tells Terraform:

What resources exist?

What did Terraform create?

What is the current condition?"

Show The Relationship

main.tf

Desired State

↓

Terraform

↓

terraform.tfstate

Current Knowledge

↓

welcome.txt

Actual Resource

Part 9 — Add More Infrastructure

Now add:

```
resource "local_file" "readme" {  
  
  filename = "README.md"  
  
  content = <<EOF  
  
# Terraform Engineering Bootcamp  
  
Academy:  
MZ-UCA Cloud Academy  
  
Instructor:  
Mthulisi Zulu  
  
Topic:  
Infrastructure as Code  
  
EOF  
  
}
```

Add:

```
resource "local_file" "policy" {  
  
  filename = "company-policy.txt"  
  
  content = <<EOF
```

COMPANY SECURITY POLICY

1. Strong passwords required.
2. Follow least privilege.
3. Protect company data.
4. Monitor systems.

EOF

}

Run Workflow Again

Format:

```
terraform fmt
```

Validate:

```
terraform validate
```

Plan:

```
terraform plan
```

Explain:

"Terraform detects that we added two new resources."

Apply:

```
terraform apply
```

Confirm:

```
yes
```

Verify:

```
dir
```

Now:

```
welcome.txt
```

```
README.md
```

```
company-policy.txt
```

Part 10 — Demonstrate Change Management

Modify:

```
welcome.txt
```

content inside Terraform.

Change:

```
Welcome to Terraform Engineering
```

to:

```
Welcome to MZ-UCA Terraform Engineering Bootcamp
```

Run:

```
terraform plan
```

Explain:

Terraform detects:

```
~ update
```

Meaning:

"The resource exists, but the desired state changed."

Apply:

```
terraform apply
```

Part 11 — Destroy Infrastructure

Explain:

"In production environments, unused infrastructure costs money.

Terraform allows controlled removal."

Preview:

```
terraform plan -destroy
```

Destroy:

```
terraform destroy
```

Confirm:

```
yes
```

Expected:

```
Destroy complete!  
Resources: 3 destroyed.
```

Final Discussion

Ask Students:

Question 1

What is Terraform?

Answer:

Terraform is an Infrastructure as Code tool used to provision and manage infrastructure.

Question 2

What is a provider?

Answer:

A provider is a plugin that allows Terraform to communicate with a platform.

Question 3

What is a resource?

Answer:

A resource is an object managed by Terraform.

Question 4

Why does Terraform need state?

Answer:

Terraform state tracks resources and allows Terraform to understand changes.

Homework Challenge

Create a Terraform project that automatically creates:

`student-profile.txt`

`career-goals.txt`

`cloud-notes.txt`

`README.md`

`project-summary.txt`

Requirements:

- ✓ Use Local Provider
- ✓ Use Terraform workflow
- ✓ Push to GitHub
- ✓ Include screenshots:
 - terraform plan
 - terraform apply

- Generated files

Instructor Closing Statement

"Today we created files.

Tomorrow we create infrastructure.

The resource changes, but the Terraform mindset remains the same.

The same workflow used today will later create:

- ☛ AWS EC2 instances
- ☛ Networks
- ☛ Databases
- ☛ Kubernetes clusters
- ☛ Enterprise cloud platforms

This is the foundation of modern Cloud Engineering."

MZ-UCA Cloud Academy

Mthulisi Zulu

Cloud Governance Architect

Building Cloud Engineers, Not Just Certification Holders.